Speech Recognition in Noise

Viva preparation guide · Whisper base.en vs Wav2Vec2 Base vs Moonshine base

1. The project in one sentence

A web app where you record yourself, type what you said, add real-world noise, and compare how well up to three pretrained speech recognisers (**Whisper base.en**, **Wav2Vec2 Base**, **Moonshine base**) understand the same noisy audio, each scored by **word error rate (WER)**.

2. What we did, step by step

Step 1: Environment

- Python 3.12 in a virtual environment (**.venv**) on a laptop CPU: 4 cores, no GPU.
- Libraries: PyTorch 2.15 (CPU), Hugging Face Transformers 5.22, Gradio 6.31, librosa, soundfile, noisereduce, JiWER, matplotlib, pandas.

Step 2: Building the noise bank (prepare_noises.py)

Noise	Source	How it was made
Rain	ESC-50 “rain” class	6 clips of 5 s each, silence trimmed, joined with 50 ms crossfades
Engine	ESC-50 “engine” class	Same method
Traffic	DEMAND STRAFFIC (busy intersection)	Channel 1, 30 s taken from 60 s in
Café babble	DEMAND PRESTO (university restaurant)	Channel 1, 30 s taken from 60 s in

- Every noise was converted to 16 kHz mono, cut to 30 s and normalised to −1 dBFS.
- Credits and licences are stored in noises.json: ESC-50 is CC BY-NC, DEMAND is CC BY-SA 3.0.
- Screaming was planned but skipped: ESC-50 has no screaming class, and Freesound clips would have to be picked and licence-checked by hand.

Step 3: Checking the models on clean speech first

- Before adding any noise, all models were tested on clean LibriSpeech clips.
- Results: Whisper 0–10%, Wav2Vec2 0–3%, Moonshine 0% WER. That proved the pipeline worked.
- It also exposed a scoring bug: Whisper writes “Mr.” but the reference says “MISTER”. It was fixed in the text normaliser.

Step 4: The shared pipeline (asr_core.py), the same code for every model

1. Load the audio as 16 kHz mono and remove DC offset.
2. Trim silence at the start and end (keeping 0.2 s), so the speech power isn’t diluted by silence.
3. Mix in the noise (section 3).
4. Send the **identical** array to each model.
5. Normalise the text and compute WER.

Step 5: The web app (app.py), built and refined over several rounds

- Record → type your transcript → choose Clean or Add noise (one or more) → preview the noisy audio → pick 1–3 models → Run.
- Results: a ranked banner and one card per model showing what you said, what the model heard, a word-by-word comparison, words right, WER and time.

Step 6: Testing

Each feature was tested by calling the app's code on real audio, and the layout was checked with browser screenshots.

3. How the noise is mixed (expect a question on this)

Basic SNR formula, which the benchmark script still uses:

$$\mathrm{SNR(dB)} = 10 \cdot \log_{10}(P_{speech} / P_{noise})$$

$$g = \sqrt{(P_{speech} / (P_{noise} \cdot 10^{SNR/10}))} \text{ and mixture} = \text{speech} + g \cdot \text{noise}$$

P is the mean squared amplitude of the signal.

What the app uses: A-weighted loudness matching

- **The problem:** at the same raw energy, traffic sounded about 12.6 dB quieter than rain, and engine about 8 dB quieter. Their energy sits at 140–260 Hz, where human hearing (and laptop speakers) are weak.
- **The fix:** each noise is scaled so its *A-weighted* power matches the speech. A-weighting is the standard curve for how loud something sounds to a person. Every noise is set at 0 dB, so it sounds about as loud as your voice.
- **Several noises:** each one is added at that level, so two noises are louder than one. The app shows the resulting overall plain SNR, e.g. rain + traffic ≈ −9 dB.
- **Other details:** each recording gets a random starting point in the noise clip (seeded, so it can be repeated). If the mixture would clip, the whole thing is scaled down, which keeps the SNR unchanged.
- **Check:** the measured SNR came out within 0.05 dB of the target for every noise.

4. The models

	Whisper base.en	Wav2Vec2 Base 960h	Moonshine base
Developer	OpenAI	Meta	Useful Sensors
Parameters	74M	94.4M	61M
Type	Encoder–decoder (sequence-to-sequence)	CNN + Transformer + CTC head	Encoder–decoder
Input	Log-Mel spectrogram	Raw waveform	Raw waveform
Training audio	680,000 h of varied web audio	960 h of clean audiobooks (LibriSpeech)	~200,000 h
Output	Cased text with punctuation	Uppercase letters, no punctuation	Cased text with punctuation

Fair-comparison settings

- **Greedy decoding for all models:** no beam search, no language model.
- **Whisper and Moonshine:** output capped at 8 tokens per second of audio + 16, which stops endless repetition loops.
- **Timing:** each model is warmed up once at start-up. Only the model's own work (forward pass and decoding) is timed, on the same CPU with 4 threads.

5. Scoring

Text normaliser, applied the same way to your transcript and to every model's output

- Lowercase everything and remove punctuation.
- Turn hyphens into spaces, but keep apostrophes inside words ("don't").
- **Numbers become words:** "4" → "four", because Whisper writes digits and Wav2Vec2 spells them out.
- Expand Mr → mister, Mrs → missus, Dr → doctor, OK → okay.

Word error rate

WER = (S + D + I) / N

S = misheard (substituted) words · D = missed (deleted) words · I = extra (inserted) words · N = number of words you said. Lower is better. WER can go above 100% when a model adds lots of extra words.

Picking the winner

- The model with the **most words right** wins. WER only breaks ties between models with the same number of words right.
- **Why:** with "hello", Whisper heard "Hello! Nice!" (1 right + 1 extra word = 100% WER) and the others heard nothing (also 100% WER). WER alone called it a tie, which was wrong.
- If nobody got a single word right, the app says "None of the models understood you" and names no winner.
- Exactly equal scores → tie, with no highlighted card.

Detecting a wrong transcript

The app warns you when all of these are true:

- at least 2 models ran and all of them produced words
- every model has WER ≥ 50% against your text
- the models agree with *each other* on at least 60% of words

That pattern means your typed text, not the models, is probably wrong.

6. What we saw in testing (examples, not final results)

- **Clean speech:** all three models were near 0% WER.
- **Café babble at about 0 dB:** Whisper 12% WER, Moonshine 47%, Wav2Vec2 76% (one LibriSpeech clip).
- **Rain + traffic at about −10 dB:** Wav2Vec2 often returns **nothing at all**. It was trained only on clean audiobooks, so in heavy noise its CTC output picks "blank" for every frame. Whisper still guesses, e.g. it heard "I am studying in college" as "I am standing here".
- **Speed on this CPU:** Wav2Vec2 and Moonshine take about 0.8 s per clip, Whisper about 2–3 s. Whisper is slower because it pads every clip to 30 s and generates text one token at a time.

These come from a handful of clips. Present them as **indicative**, not as conclusions.

7. Changes from the original proposal (say these plainly)

Proposal	What we did	Why
Whisper Tiny	Whisper **Base**	Closer in size to Wav2Vec2 (1.3× apart instead of 2.4×)
2 models	3 models; pick any 1–3	Moonshine adds a newer, efficient design
Fixed reference sentences	You type your own transcript	Lets you test any speech
SNR picker 10 / 5 / 0 dB	Fixed 0 dB, A-weighted	Equal *perceived* loudness across noises; simpler to use
Noise reduction in the app	Removed from the app (code kept in asr_core.py)	Kept the app simple
5 noises	4 (no screaming)	No licence-clean source ready
Fine-tuning	Not done (optional extension)	Not needed for the live comparison

8. Limitations (examiners like hearing these)

- **Training data differs hugely:** 680,000 h vs 960 h, so this compares whole recipes, not just architectures.
- **Small sample:** one speaker, one microphone, a handful of sentences.

- **Synthetic mixing:** noise is added after recording. Real rooms add echo, and people raise their voice in noise (the Lombard effect).
- **English only** and a narrow range of accents.
- **Whisper can hallucinate** words that were never said, and WER counts them.
- **Timing depends on the machine.**
- **The benchmark script isn't aligned with the app yet** (single noises, plain SNR, the old recordings layout), so there are no full-scale results.

9. Likely viva questions and short answers

1. Why these models?
They're different families (sequence-to-sequence vs CTC), all pretrained, and similar in size (61–94M parameters).

2. What is WER?
(Misheard + missed + extra words) ÷ words you said. Lower is better.

3. Why can WER go above 100%?
Extra words count as errors, and there's no limit to how many a model can add.

4. What is SNR? What does 0 dB mean?
The ratio of speech power to noise power; 0 dB means they're equal.

5. Why A-weighting?
Low rumbles carry energy we barely hear. Matching raw energy made traffic sound about 13 dB quieter than rain.

6. Why does Wav2Vec2 sometimes output nothing?
It's a CTC model trained only on clean audiobooks. In heavy noise it predicts "blank" for every frame.

7. Why is Whisper slower?
It pads every clip to 30 s and generates text one token at a time.

8. CTC vs sequence-to-sequence?
CTC predicts one symbol per audio frame, all at once. Seq2seq generates text step by step, like a language model, so it can guess from context but also hallucinate.

9. How do you keep it fair?
Identical audio for every model, one text normaliser, greedy decoding for all, the same CPU and thread count, and timing that covers only the model's work.

10. Why turn numbers into words?
Whisper writes "4", Wav2Vec2 writes "FOUR". Without it, correct answers would count as errors.

11. Why rank by words right and not just WER?
Short sentences: one extra word can give the same WER as getting everything wrong.

12. What if the user types the wrong transcript?
WER becomes meaningless. The app warns when the models agree with each other but not with the text.

13. Is noise reduction the same as fine-tuning?
No. Noise reduction changes the input audio; fine-tuning changes the model's weights.

14. How would you improve it?
More speakers and accents, real recorded noisy rooms, fine-tuning on noisy speech, larger models, and a full benchmark with statistics.

10. Project files

File	What it does
app.py	The Gradio web app: layout, noise preview, running the models, result cards
asr_core.py	Shared pipeline: audio loading, silence trimming, noise mixing (plain and A-weighted), the models, text normaliser, WER
prepare_noises.py	Downloads ESC-50 and DEMAND clips and builds the 30 s noise files
benchmark.py	Batch evaluation over many recordings and conditions; writes results and charts
plots.py	Charts with one fixed colour per model
models.json	The three models and their details
noises.json	Noise names, sources, licences and per-clip credits
noises/*.wav	The four prepared noise clips

Run the app: **cd D:\ASR** then **.venv\Scripts\python app.py**, and open http://127.0.0.1:7860